Go by Example

The Edited Reference

Complete and self-contained Go examples.

The original Go by Example project is by Mark McGranaghan and contributors, including Eli Bendersky. The source material is available at gobyexample.com and github.com/mmcgrana/gobyexample.

Adapted, corrected, and reformatted from the Go by Example project.
Attribution and license appear in the opening pages.

Contents

1 Hello World 1
2 Values 2
3 Variables 3
4 Constants 4
5 For 5
6 If/Else 6
7 Switch 7
8 Arrays 8
9 Slices 9
10 Maps 10
11 Functions 11
12 Multiple Return Values 12
13 Variadic Functions 13
14 Closures 14
15 Recursion 15
16 Range over Built-in Types 16
17 Pointers 17
18 Strings and Runes 18
19 Structs 19
20 Methods 20
21 Interfaces 21
22 Enums 22
23 Struct Embedding 23
24 Generics 24
25 Range over Iterators 25

26 Errors 26

27 Custom Errors 27

28 Goroutines 28

29 Channels 29

30 Channel Buffering 30

31 Channel Synchronization 31

32 Channel Directions 32

33 Select 33

34 Timeouts 34

35 Non-Blocking Channel Operations 35

36 Closing Channels 36

37 Range over Channels 37

38 Timers 38

39 Tickers 39

40 Worker Pools 40

41 WaitGroups 41

42 Rate Limiting 42

43 Atomic Counters 43

44 Mutexes 44

45 Stateful Goroutines 45

46 Sorting 46

47 Sorting by Functions 47

48 Panic 48

49 Defer 49

50 Recover 50

51 String Functions 51

52 String Formatting 52
53 Text Templates 53
54 Regular Expressions 54
55 JSON 55
56 XML 56
57 Time 57
58 Epoch 58
59 Time Formatting / Parsing 59
60 Random Numbers 60
61 Number Parsing 61
62 URL Parsing 62
63 SHA256 Hashes 63
64 Base64 Encoding 64
65 Reading Files 65
66 Writing Files 66
67 Line Filters 67
68 File Paths 68
69 Directories 69
70 Temporary Files and Directories 70
71 Embed Directive 71
72 Testing and Benchmarking 72
73 Command-Line Arguments 73
74 Command-Line Flags 74
75 Command-Line Subcommands 75
76 Environment Variables 76
77 Logging 77

78 HTTP Client 78

79 HTTP Server 79

80 TCP Server 80

81 Context 81

82 Spawning Processes 82

83 Exec’ing Processes 83

84 Signals 84

85 Exit 85

How to use this reference

Each chapter begins on a new page. The single **Example** listing in each chapter is complete Go source rather than a disconnected fragment. Save it as `main.go` and run `go run main.go`, except where a chapter explicitly describes a different file layout or a persistent server.

Every chapter has an output example directly below its code. For predictable programs, the listing shows the expected output. When results depend on goroutine scheduling, elapsed time, command-line arguments, the operating system, files, a remote service, or an external request, the listing shows *one representative run*; a brief note explains what may vary. Terminal output can combine standard output and standard error when stated. A modern Go compiler is recommended; the iterator chapter requires Go 1.23 or later.

Quick reading guide. Declarations at package scope appear before `main`. Imports appear only when needed. The point of a chapter is the rule immediately above its example; the program supplies the runnable context needed to understand that rule.

Attribution and license

The original Go by Example project is by Mark McGranaghan and contributors, including Eli Bendersky. The source material is available at gobyexample.com and github.com/mmcgrana/gobyexample.

The original work is licensed under Creative Commons Attribution 3.0 Unported (CC BY 3.0). This edition is an adaptation: the presentation, explanatory text, and sample programs have been revised substantially to make the material useful as a compact reference. The original authors do not endorse this adaptation.

1 Hello World

A Go program starts in package main and executes func main. The fmt package prints formatted values.

Example

```
package main

import "fmt"

func main() {
    fmt.Println("hello world")
}
```

Output

```
hello world
```

2 Values

Go has strings, integers, floating-point values, and booleans. Operators act on compatible types.

Example

```
package main

import "fmt"

func main() {
    fmt.Println("go" + "lang")
    fmt.Println(1 + 1)
    fmt.Println(7.0 / 3.0)
    fmt.Println(true && false)
    fmt.Println(true || false)
    fmt.Println(!true)
}
```

Output

```
golang
2
2.3333333333333335
false
true
false
```

3 Variables

Use var for an explicit declaration and := for a short declaration inside a function. Uninitialized variables receive their zero value.

Example

```
package main

import "fmt"

func main() {
    var name string = "Go"
    count := 3
    var ready bool
    fmt.Println(name, count, ready)
    count = 4
    fmt.Println(count)
}
```

Output

```
Go 3 false
4
```

4 Constants

Constants are immutable. Untyped numeric constants can retain precision until a context gives them a type.

Example

```
package main

import (
    "fmt"
    "math"
)

const message = "constant"
const big = 3e10

func main() {
    fmt.Println(message)
    fmt.Println(int64(big))
    fmt.Printf("%.2f\n", math.Sin(math.Pi/2))
}
```

Output

```
constant
30000000000
1.00
```

5 For

for covers counted iteration, condition-only loops, and iteration over collections. break exits a loop; continue skips to the next iteration.

Example

```
package main

import "fmt"

func main() {
    for i := 0; i < 3; i++ {
        fmt.Println(i)
    }
    n := 3
    for n > 0 {
        n--
    }
    fmt.Println("done")
}
```

Output

```
0
1
2
done
```

6 If/Else

if conditions need no parentheses, but braces are mandatory. A short statement before the condition scopes a variable to the entire if/else chain.

Example

```
package main

import "fmt"

func main() {
    if n := 9; n < 0 {
        fmt.Println("negative")
    } else if n < 10 {
        fmt.Println("one digit")
    } else {
        fmt.Println("many digits")
    }
}
```

Output

```
one digit
```

7 Switch

switch chooses a branch without implicit fallthrough. Multiple values can share a case; a switch with no expression acts like a chain of conditions.

Example

```
package main

import "fmt"

func main() {
    day := "Saturday"
    switch day {
    case "Saturday", "Sunday":
        fmt.Println("weekend")
    default:
        fmt.Println("weekday")
    }
}
```

Output

```
weekend
```

8 Arrays

Arrays have fixed length, and that length is part of their type. Indexes start at zero; assigning an array copies its elements.

Example

```
package main

import "fmt"

func main() {
    a := [3]int{1, 2, 3}
    b := a
    b[0] = 99
    fmt.Println(a)
    fmt.Println(b)
    fmt.Println(len(a))
}
```

Output

```
[1 2 3]
[99 2 3]
3
```

9 Slices

Slices are dynamically sized views over arrays. append may allocate new storage, so use its return value.

Example

```
package main

import "fmt"

func main() {
    a := []int{1, 2, 3}
    b := a[1:]
    b[0] = 20
    a = append(a, 4)
    fmt.Println(a)
    fmt.Println(len(a), cap(a))
}
```

Output

```
[1 20 3 4]
4 6
```

10 Maps

Maps associate comparable keys with values. A lookup can return both the value and a boolean indicating whether the key exists.

Example

```
package main

import "fmt"

func main() {
    ages := map[string]int{"Ada": 36}
    ages["Bob"] = 41
    age, ok := ages["Ada"]
    fmt.Println(age, ok)
    delete(ages, "Bob")
    _, ok = ages["Bob"]
    fmt.Println(ok)
}
```

Output

```
36 true
false
```

11 Functions

Functions name their parameters and result types. A function can return a value or return nothing.

Example

```
package main

import "fmt"

func add(a, b int) int {
    return a + b
}

func main() {
    fmt.Println(add(2, 3))
}
```

Output

```
5
```

12 Multiple Return Values

A function may return multiple results. A common pattern returns a value and an error.

Example

```
package main

import "fmt"

func divide(a, b int) (int, bool) {
    if b == 0 {
        return 0, false
    }
    return a / b, true
}

func main() {
    n, ok := divide(12, 3)
    fmt.Println(n, ok)
}
```

Output

```
4 true
```

13 Variadic Functions

A variadic parameter accepts zero or more arguments. Inside the function, it behaves as a slice.

Example

```
package main

import "fmt"

func sum(nums ...int) int {
    total := 0
    for _, n := range nums {
        total += n
    }
    return total
}

func main() {
    values := []int{1, 2, 3}
    fmt.Println(sum(values...))
}
```

Output

```
6
```

14 Closures

A closure retains access to variables in its surrounding function, even after that outer function returns.

Example

```
package main

import "fmt"

func counter() func() int {
    n := 0
    return func() int {
        n++
        return n
    }
}

func main() {
    next := counter()
    fmt.Println(next(), next(), next())
}
```

Output

```
1 2 3
```

15 Recursion

Recursive functions call themselves and must have a base case to terminate.

Example

```
package main

import "fmt"

func factorial(n int) int {
    if n <= 1 {
        return 1
    }
    return n * factorial(n-1)
}

func main() {
    fmt.Println(factorial(5))
}
```

Output

```
120
```

16 Range over Built-in Types

range iterates over slices, arrays, strings, maps, and other iterable values. An underscore discards an unused index or value.

Example

```
package main

import "fmt"

func main() {
    nums := []int{2, 3, 4}
    total := 0
    for _, n := range nums {
        total += n
    }
    fmt.Println(total)
    for i, n := range nums {
        fmt.Printf("%d:%d ", i, n)
    }
    fmt.Println()
}
```

Output

```
9
0:2 1:3 2:4
```

17 Pointers

A pointer holds the address of a variable. Passing a pointer allows a function to modify the original value.

Example

```
package main

import "fmt"

func zero(p *int) {
    *p = 0
}

func main() {
    n := 7
    zero(&n)
    fmt.Println(n)
}
```

Output

```
0
```

18 Strings and Runes

A string stores bytes, usually UTF-8 encoded text. range over a string decodes Unicode code points (runes).

Example

```
package main

import (
    "fmt"
    "unicode/utf8"
)

func main() {
    text := "Go! □□"
    fmt.Println(len(text))
    fmt.Println(utf8.RuneCountInString(text))
    for _, r := range "Go" {
        fmt.Printf("%c ", r)
    }
    fmt.Println()
}
```

Output

```
10
6
G o
```

19 Structs

Structs group fields into a named type. Construct them using field names to avoid order-dependent initialization.

Example

```
package main

import "fmt"

type Person struct {
    Name string
    Age  int
}

func main() {
    p := Person{Name: "Ada", Age: 36}
    p.Age++
    fmt.Println(p.Name, p.Age)
}
```

Output

```
Ada 37
```

20 Methods

A method is a function with a receiver. Pointer receivers can modify a value and avoid copying large structs.

Example

```
package main

import "fmt"

type Rectangle struct {
    Width, Height int
}

func (r Rectangle) Area() int {
    return r.Width * r.Height
}

func (r *Rectangle) Scale(n int) {
    r.Width *= n
    r.Height *= n
}

func main() {
    r := Rectangle{3, 4}
    r.Scale(2)
    fmt.Println(r.Area())
}
```

Output

```
48
```

21 Interfaces

An interface describes behavior via a method set. A concrete type satisfies an interface implicitly by providing those methods.

Example

```
package main

import "fmt"

type Speaker interface {
    Speak() string
}

type Dog struct{}

func (Dog) Speak() string { return "woof" }

func announce(s Speaker) {
    fmt.Println(s.Speak())
}

func main() {
    announce(Dog{})
}
```

Output

```
woof
```

22 Enums

Go has no built-in enum declaration. Named types plus iota provide readable sets of related constants.

Example

```
package main

import "fmt"

type State int

const (
    Idle State = iota
    Running
    Stopped
)

func main() {
    s := Running
    fmt.Println(s == Running, s)
}
```

Output

```
true 1
```

23 Struct Embedding

Embedding promotes fields and methods of an embedded type while retaining normal composition.

Example

```
package main

import "fmt"

type Base struct{ ID int }

func (b Base) Label() string { return fmt.Sprint(b.ID) }

type Job struct {
    Base
    Name string
}

func main() {
    j := Job{Base: Base{ID: 7}, Name: "build"}
    fmt.Println(j.ID, j.Label(), j.Name)
}
```

Output

```
7 7 build
```

24 Generics

Type parameters make functions work with multiple types while preserving static type checking. Constraints restrict permitted operations.

Example

```
package main

import "fmt"

func Index[T comparable](items []T, want T) int {
    for i, v := range items {
        if v == want {
            return i
        }
    }
    return -1
}

func main() {
    fmt.Println(Index([]string{"a", "b"}, "b"))
}
```

Output

```
1
```

25 Range over Iterators

Iterator functions can yield a sequence to range. The yield callback returns false when the consumer stops early (Go 1.23+).

Example

```
package main

import "fmt"

func Count(n int) func(func(int) bool) {
    return func(yield func(int) bool) {
        for i := 0; i < n; i++ {
            if !yield(i) {
                return
            }
        }
    }
}

func main() {
    for n := range Count(3) {
        fmt.Println(n)
    }
}
```

Output

```
0
1
2
```

26 Errors

Go errors are ordinary values. Check err explicitly and add useful context with fmt.Errorf and %w.

Example

```
package main

import (
    "errors"
    "fmt"
)

func load(name string) error {
    if name == "" {
        return errors.New("empty name")
    }
    return nil
}

func main() {
    if err := load(""); err != nil {
        fmt.Println("load failed:", err)
    }
}
```

Output

```
load failed: empty name
```

27 Custom Errors

Custom error types can carry structured context. errors.As finds a matching error type through wrapping.

Example

```
package main

import (
    "errors"
    "fmt"
)

type StatusError struct{ Code int }

func (e *StatusError) Error() string {
    return fmt.Sprintf("status %d", e.Code)
}

func main() {
    err := fmt.Errorf("request: %w", &StatusError{404})
    var target *StatusError
    if errors.As(err, &target) {
        fmt.Println(target.Code)
    }
}
```

Output

```
404
```

28 Goroutines

A goroutine runs a function concurrently. Synchronize completion explicitly; sleeping is not a reliable synchronization technique.

Example

```
package main

import (
    "fmt"
    "sync"
)

func main() {
    var wg sync.WaitGroup
    wg.Add(1)
    go func() {
        defer wg.Done()
        fmt.Println("from goroutine")
    }()
    wg.Wait()
}
```

Output

```
from goroutine
```

29 Channels

Channels transfer typed values between goroutines. An unbuffered channel pairs a sender with a receiver.

Example

```
package main

import "fmt"

func main() {
    ch := make(chan string)
    go func() { ch <- "hello" }()
    fmt.Println(<-ch)
}
```

Output

```
hello
```

30 Channel Buffering

Buffered channels hold a limited number of queued values. A send blocks only when the buffer is full.

Example

```
package main

import "fmt"

func main() {
    ch := make(chan int, 2)
    ch <- 10
    ch <- 20
    fmt.Println(<-ch, <-ch)
}
```

Output

```
10 20
```

31 Channel Synchronization

A channel can signal that work finished. Closing a channel is a common broadcast signal when no more values will arrive.

Example

```
package main

import "fmt"

func main() {
    done := make(chan struct{})
    go func() {
        fmt.Println("working")
        close(done)
    }()
    <-done
    fmt.Println("finished")
}
```

Output

```
working
finished
```

32 Channel Directions

Channel direction annotations express intent and let the compiler reject mistaken sends or receives.

Example

```
package main

import "fmt"

func send(ch chan<- string) {
    ch <- "ping"
}

func receive(ch <-chan string) {
    fmt.Println(<-ch)
}

func main() {
    ch := make(chan string)
    go send(ch)
    receive(ch)
}
```

Output

```
ping
```

33 Select

select waits for one ready channel operation. A default branch makes the select non-blocking.

Example

```
package main

import "fmt"

func main() {
    ch := make(chan int, 1)
    ch <- 42
    select {
    case n := <-ch:
        fmt.Println(n)
    default:
        fmt.Println("not ready")
    }
}
```

Output

```
42
```

34 Timeouts

Use context deadlines to bound operations instead of waiting forever. Always call cancel to release associated resources.

Example

```
package main

import (
    "context"
    "fmt"
    "time"
)

func main() {
    ctx, cancel := context.WithTimeout(context.Background(), time.Millisecond)
    defer cancel()
    <-ctx.Done()
    fmt.Println(ctx.Err())
}
```

Output

```
context deadline exceeded
```

35 Non-Blocking Channel Operations

A select with default probes channels without waiting. That is useful for optional work, but not as a substitute for proper synchronization.

Example

```
package main

import "fmt"

func main() {
    ch := make(chan string)
    select {
    case msg := <-ch:
        fmt.Println(msg)
    default:
        fmt.Println("no message")
    }
}
```

Output

```
no message
```

36 Closing Channels

Only the sender should normally close a channel. A receiver can test whether a receive succeeded with the two-result form.

Example

```
package main

import "fmt"

func main() {
    jobs := make(chan int, 2)
    jobs <- 1
    jobs <- 2
    close(jobs)
    for {
        job, ok := <-jobs
        if !ok {
            break
        }
        fmt.Println(job)
    }
}
```

Output

```
1
2
```

37 Range over Channels

range ch receives until the channel is closed. It is the idiomatic way to consume a stream with an explicit end.

Example

```
package main

import "fmt"

func main() {
    ch := make(chan string, 2)
    ch <- "one"
    ch <- "two"
    close(ch)
    for msg := range ch {
        fmt.Println(msg)
    }
}
```

Output

```
one
two
```

38 Timers

time.NewTimer fires once. Stop or drain unused timers when appropriate to avoid unnecessary work.

Example

```
package main

import (
    "fmt"
    "time"
)

func main() {
    timer := time.NewTimer(10 * time.Millisecond)
    defer timer.Stop()
    <-timer.C
    fmt.Println("timer fired")
}
```

Output

```
timer fired
```

39 Tickers

A ticker produces periodic events. Always stop a ticker when you are finished with it.

Example

```
package main

import (
    "fmt"
    "time"
)

func main() {
    ticker := time.NewTicker(10 * time.Millisecond)
    defer ticker.Stop()
    for i := 0; i < 2; i++ {
        <-ticker.C
        fmt.Println("tick")
    }
}
```

Output

```
tick
tick
```

40 Worker Pools

A worker pool caps concurrency by running a fixed number of workers that consume tasks from a shared channel.

Example

```
package main

import (
    "fmt"
    "sync"
)

func worker(jobs <-chan int, results chan<- int, wg *sync.WaitGroup) {
    defer wg.Done()
    for n := range jobs {
        results <- n * n
    }
}

func main() {
    jobs := make(chan int, 3)
    results := make(chan int, 3)
    var wg sync.WaitGroup
    wg.Add(2)
    for i := 0; i < 2; i++ {
        go worker(jobs, results, &wg)
    }
    for _, n := range []int{2, 3, 4} {
        jobs <- n
    }
    close(jobs)
    wg.Wait()
    close(results)
    for n := range results {
        fmt.Println(n)
    }
}
```

Output

```
4
9
16
```

Example run. All three squares are printed, but worker scheduling can change their order.

41 WaitGroups

sync.WaitGroup coordinates completion of a known number of goroutines. Call Add before starting work, and Done exactly once per task.

Example

```
package main

import (
    "fmt"
    "sync"
)

func main() {
    var wg sync.WaitGroup
    for i := 0; i < 3; i++ {
        wg.Add(1)
        go func(n int) {
            defer wg.Done()
            fmt.Print(n, " ")
        }(i)
    }
    wg.Wait()
    fmt.Println()
}
```

Output

```
2 0 1
```

Example run. The goroutines may print the numbers in any order, depending on scheduling.

42 Rate Limiting

A time.Ticker can pace requests. For bursting workloads, a token-bucket limiter uses a buffered token channel.

Example

```
package main

import (
    "fmt"
    "time"
)

func main() {
    limiter := time.NewTicker(10 * time.Millisecond)
    defer limiter.Stop()
    for i := 1; i <= 3; i++ {
        <-limiter.C
        fmt.Println("request", i)
    }
}
```

Output

```
request 1
request 2
request 3
```

43 Atomic Counters

Atomic variables provide lock-free operations on individual values. Use sync/atomic types instead of a shared unsynchronized integer.

Example

```
package main

import (
    "fmt"
    "sync"
    "sync/atomic"
)

func main() {
    var count atomic.Int64
    var wg sync.WaitGroup
    for i := 0; i < 100; i++ {
        wg.Add(1)
        go func() {
            defer wg.Done()
            count.Add(1)
        }()
    }
    wg.Wait()
    fmt.Println(count.Load())
}
```

Output

```
100
```

44 Mutexes

A mutex protects an invariant spanning one or more variables. Lock around both reading and writing shared state.

Example

```
package main

import (
    "fmt"
    "sync"
)

type Counter struct {
    mu sync.Mutex
    n  int
}

func (c *Counter) Inc() {
    c.mu.Lock()
    defer c.mu.Unlock()
    c.n++
}

func main() {
    c := &Counter{}
    var wg sync.WaitGroup
    for i := 0; i < 10; i++ {
        wg.Add(1)
        go func() { defer wg.Done(); c.Inc() }()
    }
    wg.Wait()
    fmt.Println(c.n)
}
```

Output

```
10
```

45 Stateful Goroutines

Another concurrency model assigns ownership of mutable state to one goroutine; requests arrive via channels.

Example

```
package main

import "fmt"

type request struct {
    delta int
    reply chan int
}

func main() {
    requests := make(chan request)
    go func() {
        total := 0
        for req := range requests {
            total += req.delta
            req.reply <- total
        }
    }()
    reply := make(chan int)
    requests <- request{delta: 5, reply: reply}
    fmt.Println(<-reply)
    close(requests)
}
```

Output

```
5
```

46 Sorting

The slices package provides type-safe sorting and membership functions. Sorting changes the slice in place.

Example

```
package main

import (
    "fmt"
    "slices"
)

func main() {
    nums := []int{3, 1, 2}
    slices.Sort(nums)
    fmt.Println(nums)
    fmt.Println(slices.IsSorted(nums))
}
```

Output

```
[1 2 3]
true
```

47 Sorting by Functions

Use slices.SortFunc and a comparison function to order records by a specific field. cmp.Compare handles most ordered values.

Example

```
package main

import (
    "cmp"
    "fmt"
    "slices"
)

type Person struct {
    Name string
    Age  int
}

func main() {
    people := []Person{{"Bob", 40}, {"Ada", 36}}
    slices.SortFunc(people, func(a, b Person) int {
        return cmp.Compare(a.Age, b.Age)
    })
    fmt.Println(people)
}
```

Output

```
[{Ada 36} {Bob 40}]
```

48 Panic

panic is for unrecoverable programming failures, not routine errors. Deferred functions still run as the stack unwinds.

Example

```
package main

import "fmt"

func main() {
    defer fmt.Println("cleanup")
    fmt.Println("starting")
    panic("unexpected failure")
}
```

Output

```
starting
cleanup
panic: unexpected failure
... stack trace ...
```

Abbreviated combined stdout/stderr. The panic prints a runtime stack trace and exits with a nonzero status; the stack details and source line numbers vary.

49 Defer

defer schedules a call for when the surrounding function returns. Deferred calls run in last-in, first-out order.

Example

```
package main

import "fmt"

func main() {
    defer fmt.Println("first deferred")
    defer fmt.Println("second deferred")
    fmt.Println("body")
}
```

Output

```
body
second deferred
first deferred
```

50 Recover

recover can intercept a panic only when called from a deferred function in the same goroutine. It should be used at carefully chosen boundaries.

Example

```
package main

import "fmt"

func safeCall() {
    defer func() {
        if v := recover(); v != nil {
            fmt.Println("recovered:", v)
        }
    }()
    panic("boom")
}

func main() {
    safeCall()
    fmt.Println("still running")
}
```

Output

```
recovered: boom
still running
```

51 String Functions

strings provides helpers for matching, trimming, splitting, and replacing text. Most operations return a new string.

Example

```go
package main

import (
    "fmt"
    "strings"
)

func main() {
    fmt.Println(strings.Contains("golang", "lang"))
    fmt.Println(strings.ToUpper("go"))
    fmt.Println(strings.Join([]string{"a", "b"}, ","))
}
```

Output

```
true
GO
a,b
```

52 String Formatting

fmt.Printf formats values with verbs such as %d (integer), %s (string), %v (general), and %.2f (two decimal places).

Example

```
package main

import "fmt"

func main() {
    name, count, price := "Ada", 3, 2.5
    fmt.Printf("%s: %d items at $%.2f\n", name, count, price)
    fmt.Printf("%T\n", count)
}
```

Output

```
Ada: 3 items at $2.50
int
```

53 Text Templates

text/template separates reusable text layout from data and handles escaping according to its template language (unlike html/template).

Example

```
package main

import (
    "os"
    "text/template"
)

func main() {
    t := template.Must(template.New("greet").Parse("Hello, {{.Name}}!\n"))
    data := struct{ Name string }{Name: "Ada"}
    if err := t.Execute(os.Stdout, data); err != nil {
        panic(err)
    }
}
```

Output

```
Hello, Ada!
```

54 Regular Expressions

The regexp package uses RE2 syntax and provides matching, capture, and replacement operations without backtracking explosions.

Example

```
package main

import (
    "fmt"
    "regexp"
)

func main() {
    re := regexp.MustCompile(`[a-z]+`)
    fmt.Println(re.FindString("123abc456"))
    fmt.Println(re.ReplaceAllString("one TWO three", "x"))
}
```

Output

```
abc
x TWO x
```

55 JSON

encoding/json marshals exported struct fields. Decode into a typed struct when the schema is known and handle errors explicitly.

Example

```
package main

import (
    "encoding/json"
    "fmt"
)

type Person struct {
    Name string `json:"name"`
    Age  int    `json:"age"`
}

func main() {
    b, err := json.Marshal(Person{Name: "Ada", Age: 36})
    if err != nil {
        panic(err)
    }
    fmt.Println(string(b))
    var p Person
    if err := json.Unmarshal(b, &p); err != nil {
        panic(err)
    }
    fmt.Println(p.Name)
}
```

Output

```
{"name":"Ada","age":36}
Ada
```

56 XML

encoding/xml maps XML elements and attributes to Go structs. Use field tags when XML element names differ from Go field names.

Example

```
package main

import (
    "encoding/xml"
    "fmt"
)

type Item struct {
    XMLName xml.Name `xml:"item"`
    ID      int      `xml:"id,attr"`
    Title   string   `xml:"title"`
}

func main() {
    b, err := xml.Marshal(Item{ID: 7, Title: "Go"})
    if err != nil {
        panic(err)
    }
    fmt.Println(string(b))
}
```

Output

```
<item id="7"><title>Go</title></item>
```

57 Time

time.Time represents an instant; time.Duration represents elapsed time. Prefer Add and Sub for arithmetic.

Example

```
package main

import (
    "fmt"
    "time"
)

func main() {
    start := time.Date(2026, 1, 2, 3, 4, 5, 0, time.UTC)
    end := start.Add(90 * time.Minute)
    fmt.Println(end.Format(time.RFC3339))
    fmt.Println(end.Sub(start))
}
```

Output

```
2026-01-02T04:34:05Z
1h30m0s
```

58 Epoch

Unix timestamps count elapsed seconds or smaller units from 1970-01-01 UTC. Do not confuse seconds, milliseconds, and nanoseconds.

Example

```
package main

import (
    "fmt"
    "time"
)

func main() {
    t := time.Unix(0, 0).UTC()
    fmt.Println(t.Format(time.RFC3339))
    fmt.Println(t.Unix())
}
```

Output

```
1970-01-01T00:00:00Z
0
```

59 Time Formatting / Parsing

Go uses a reference timestamp rather than printf-style placeholders: 2006-01-02 15:04:05.

Example

```
package main

import (
    "fmt"
    "time"
)

func main() {
    t, err := time.Parse("2006-01-02", "2026-10-21")
    if err != nil {
        panic(err)
    }
    fmt.Println(t.Format("Jan 2, 2006"))
    fmt.Println(t.Format(time.RFC3339))
}
```

Output

```
Oct 21, 2026
2026-10-21T00:00:00Z
```

60 Random Numbers

math/rand/v2 generates pseudorandom values for simulations and sampling. Do not use it for secrets; use crypto/rand instead.

Example

```
package main

import (
    "fmt"
    "math/rand/v2"
)

func main() {
    n := rand.IntN(10) // 0 through 9
    fmt.Println(n >= 0 && n < 10)
}
```

Output

```
true
```

61 Number Parsing

strconv converts strings into numeric types and returns errors when input is invalid.

Example

```
package main

import (
    "fmt"
    "strconv"
)

func main() {
    n, err := strconv.Atoi("42")
    if err != nil {
        panic(err)
    }
    fmt.Println(n + 1)
    fmt.Println(strconv.FormatBool(true))
}
```

Output

```
43
true
```

62 URL Parsing

net/url parses URL components, and url.Values encodes query parameters. Never concatenate untrusted query values by hand.

Example

```
package main

import (
    "fmt"
    "net/url"
)

func main() {
    u, err := url.Parse("https://example.com/search?q=go%20lang")
    if err != nil {
        panic(err)
    }
    fmt.Println(u.Host)
    fmt.Println(u.Query().Get("q"))
}
```

Output

```
example.com
go lang
```

63 SHA256 Hashes

SHA-256 maps bytes to a fixed-size digest. It is a hash, not encryption; do not use plain SHA-256 to store passwords.

Example

```
package main

import (
    "crypto/sha256"
    "fmt"
)

func main() {
    digest := sha256.Sum256([]byte("hello"))
    fmt.Printf("%x\n", digest)
}
```

Output

```
2cf24dba5fb0a30e26e83b2ac5b9e29e1b161e5c1fa7425e73043362938b9824
```

64 Base64 Encoding

Base64 transforms bytes into printable text; it does not provide confidentiality. Choose the URL-safe encoding for URL components.

Example

```
package main

import (
    "encoding/base64"
    "fmt"
)

func main() {
    encoded := base64.StdEncoding.EncodeToString([]byte("hello"))
    decoded, err := base64.StdEncoding.DecodeString(encoded)
    if err != nil {
        panic(err)
    }
    fmt.Println(encoded)
    fmt.Println(string(decoded))
}
```

Output

```
aGVsbG8=
hello
```

65 Reading Files

os.ReadFile is convenient for small files. For streaming or large files, prefer bufio.Scanner or buffered readers.

Example

```
package main

import (
    "fmt"
    "os"
)

func main() {
    path := "example.txt"
    if err := os.WriteFile(path, []byte("hello\n"), 0600); err != nil {
        panic(err)
    }
    defer os.Remove(path)
    data, err := os.ReadFile(path)
    if err != nil {
        panic(err)
    }
    fmt.Print(string(data))
}
```

Output

```
hello
```

66 Writing Files

os.WriteFile atomically replaces neither the file nor its directory entry: for crash-safe updates, write a temporary file and rename it.

Example

```
package main

import (
    "fmt"
    "os"
)

func main() {
    path := "out.txt"
    err := os.WriteFile(path, []byte("saved\n"), 0600)
    if err != nil {
        panic(err)
    }
    defer os.Remove(path)
    fmt.Println("wrote", path)
}
```

Output

```
wrote out.txt
```

67 Line Filters

A bufio.Scanner reads input line by line. Check scanner.Err after iteration; increase the buffer for very long lines.

Example

```
package main

import (
    "bufio"
    "fmt"
    "strings"
)

func main() {
    scanner := bufio.NewScanner(strings.NewReader("one\ntwo\n"))
    for scanner.Scan() {
        fmt.Println(strings.ToUpper(scanner.Text()))
    }
    if err := scanner.Err(); err != nil {
        panic(err)
    }
}
```

Output

```
ONE
TWO
```

68 File Paths

filepath.Join builds OS-native paths; filepath.Base and filepath.Ext inspect components without manually splitting separators.

Example

```
package main

import (
    "fmt"
    "path/filepath"
)

func main() {
    path := filepath.Join("docs", "notes.txt")
    fmt.Println(path)
    fmt.Println(filepath.Base(path))
    fmt.Println(filepath.Ext(path))
}
```

Output

```
docs/notes.txt
notes.txt
.txt
```

69 Directories

os.MkdirAll creates nested directories. os.ReadDir lists entries; filepath.WalkDir visits a whole tree with error handling.

Example

```
package main

import (
    "fmt"
    "os"
    "path/filepath"
)

func main() {
    root, err := os.MkdirTemp("", "tree-*")
    if err != nil {
        panic(err)
    }
    defer os.RemoveAll(root)
    path := filepath.Join(root, "a", "b")
    if err := os.MkdirAll(path, 0755); err != nil {
        panic(err)
    }
    entries, err := os.ReadDir(filepath.Join(root, "a"))
    if err != nil {
        panic(err)
    }
    for _, e := range entries {
        fmt.Println(e.Name())
    }
}
```

Output

```
b
```

70 Temporary Files and Directories

os.CreateTemp and os.MkdirTemp choose unique names safely. Remove temporary files and directories when no longer needed.

Example

```
package main

import (
    "fmt"
    "os"
)

func main() {
    f, err := os.CreateTemp("", "demo-*.txt")
    if err != nil {
        panic(err)
    }
    defer os.Remove(f.Name())
    defer f.Close()
    fmt.Println("temporary file created:", f.Name() != "")
}
```

Output

```
temporary file created: true
```

71 Embed Directive

The //go:embed directive includes a file at compile time. The named file must exist relative to the package directory.

Example

```
package main

import (
    _ "embed"
    "fmt"
)

//go:embed message.txt
var message string

func main() {
    fmt.Print(message)
}
```

Create a file named `message.txt` in the same directory containing `hello from a file` followed by a newline before building the program.

Output

```
hello from a file
```

72 Testing and Benchmarking

Tests live in files ending with _test.go. A test function receives *testing.T; benchmarks receive *testing.B.

Example: calc.go

```
package calc

func Add(a, b int) int { return a + b }
```

Put the function above in `calc.go` and the following test in `calc_test.go`:

Example: calc_test.go

```
package calc

import "testing"

func TestAdd(t *testing.T) {
    if got := Add(2, 3); got != 5 {
        t.Fatalf("Add(2, 3) = %d; want 5", got)
    }
}

func BenchmarkAdd(b *testing.B) {
    for i := 0; i < b.N; i++ {
        Add(2, 3)
    }
}
```

Output

```
=== RUN   TestAdd
--- PASS: TestAdd (0.00s)
BenchmarkAdd-5  189387514  1.841 ns/op
PASS
ok  example/calc  0.453s
```

Example using `go test -v -bench=.` (tests and benchmarks). Benchmark iterations, nanoseconds per operation, CPU-dependent suffixes, package path, and elapsed times vary. Tests report `PASS` when assertions succeed.

73 Command-Line Arguments

os.Args contains the executable path as element zero and then command-line arguments. Validate the count before indexing.

Example

```
package main

import (
    "fmt"
    "os"
)

func main() {
    if len(os.Args) < 2 {
        fmt.Println("usage: app NAME")
        return
    }
    fmt.Println("hello", os.Args[1])
}
```

Output

```
hello Ada
```

Example using `go run main.go Ada`. The greeting changes with the first argument. With no argument, the program prints usage information instead.

74 Command-Line Flags

The flag package parses typed command-line options. Define flags before calling flag.Parse.

Example

```
package main

import (
    "flag"
    "fmt"
)

func main() {
    name := flag.String("name", "world", "name to greet")
    flag.Parse()
    fmt.Println("hello", *name)
}
```

Output

```
hello Ada
```

Example using `go run main.go -name Ada`. The flag controls the name; omitting it prints `hello world`.

75 Command-Line Subcommands

Subcommands can use separate flag.FlagSet values. Check the command first, then parse only its arguments.

Example

```
package main

import (
    "flag"
    "fmt"
    "os"
)

func main() {
    if len(os.Args) < 2 || os.Args[1] != "greet" {
        fmt.Println("usage: app greet [-name NAME]")
        return
    }
    greet := flag.NewFlagSet("greet", flag.ExitOnError)
    name := greet.String("name", "world", "name")
    greet.Parse(os.Args[2:])
    fmt.Println("hello", *name)
}
```

Output

```
hello Ada
```

Example using `go run main.go greet -name Ada`. The output depends on the subcommand and flags; invalid or missing subcommands print usage information.

76 Environment Variables

Use os.LookupEnv when you need to distinguish an unset environment variable from an empty one.

Example

```
package main

import (
    "fmt"
    "os"
)

func main() {
    os.Setenv("APP_MODE", "test")
    mode, ok := os.LookupEnv("APP_MODE")
    fmt.Println(mode, ok)
}
```

Output

```
test true
```

77 Logging

log/slog provides structured logs with named fields and pluggable handlers. Avoid secrets in log output.

Example

```
package main

import (
    "log/slog"
    "os"
)

func main() {
    log := slog.New(slog.NewTextHandler(os.Stdout, nil))
    log.Info("request handled", "method", "GET", "status", 200)
}
```

Output

```
time=2026-10-21T14:21:00Z level=INFO msg="request handled" method=GET status=200
```

Example run. The timestamp changes for each invocation. Log fields and formatting depend on the configured handler.

78 HTTP Client

http.Client makes requests. Always close response bodies and check status codes before treating a response as successful.

Example

```
package main

import (
    "fmt"
    "net/http"
    "time"
)

func main() {
    client := &http.Client{Timeout: 5 * time.Second}
    resp, err := client.Get("https://example.com")
    if err != nil {
        panic(err)
    }
    defer resp.Body.Close()
    fmt.Println("status:", resp.StatusCode)
}
```

Output

```
status: 200
```

Example response. The status code depends on the remote server; DNS, TLS, timeouts, or connectivity can cause the request to fail instead.

79 HTTP Server

An HTTP handler writes a response through http.ResponseWriter. ServeMux routes method/path patterns to handlers (Go 1.22+).

Example

```
package main

import (
    "fmt"
    "log"
    "net/http"
)

func main() {
    mux := http.NewServeMux()
    mux.HandleFunc("GET /hello", func(w http.ResponseWriter, r *http.Request) {
        fmt.Fprintln(w, "hello")
    })
    log.Fatal(http.ListenAndServe(":8080", mux))
}
```

Output

```
hello
```

Example HTTP response body. Run the server, then request `GET /hello` on port 8080 from another terminal or browser. The server keeps running; it does not print this body to its own terminal.

80 TCP Server

net.Listen accepts incoming TCP connections. Handle each client separately and close connections when finished.

Example

```
package main

import (
    "bufio"
    "fmt"
    "log"
    "net"
)

func main() {
    ln, err := net.Listen("tcp", ":8080")
    if err != nil {
        log.Fatal(err)
    }
    defer ln.Close()
    for {
        conn, err := ln.Accept()
        if err != nil {
            log.Print(err)
            continue
        }
        go func(c net.Conn) {
            defer c.Close()
            text, err := bufio.NewReader(c).ReadString('\n')
            if err == nil {
                fmt.Fprint(c, text)
            }
        }(conn)
    }
}
```

Output

```
ping
```

Example client response. After connecting to port 8080, a client sending `ping` followed by a newline receives that line back. The server runs until stopped; only the first line of each connection is echoed.

81 Context

context.Context carries cancellation and deadlines through a call chain. Pass it as the first argument rather than storing it on a struct.

Example

```
package main

import (
    "context"
    "fmt"
    "time"
)

func main() {
    ctx, cancel := context.WithCancel(context.Background())
    cancel()
    select {
    case <-ctx.Done():
        fmt.Println(ctx.Err())
    case <-time.After(time.Second):
        fmt.Println("finished")
    }
}
```

Output

```
context canceled
```

82 Spawning Processes

os/exec starts processes without invoking a shell by default. Avoid constructing shell command strings from user input.

Example

```
package main

import (
    "fmt"
    "os/exec"
)

func main() {
    out, err := exec.Command("go", "version").Output()
    if err != nil {
        panic(err)
    }
    fmt.Print(string(out))
}
```

Output

```
go version go1.23.2 linux/amd64
```

Example run. The Go version, operating system, and architecture match the toolchain installed on the host and therefore vary.

83 Exec'ing Processes

syscall.Exec replaces the current process on Unix. It does not return on success and is generally inappropriate for portable applications.

Example

```
package main

import (
    "os"
    "os/exec"
    "syscall"
)

func main() {
    binary, err := exec.LookPath("ls")
    if err != nil {
        panic(err)
    }
    if err := syscall.Exec(binary, []string{"ls", "-l"}, os.Environ()); err != nil
    {
        panic(err)
    }
}
```

Output

```
total 4
-rw-r--r-- 1 user user 237 Oct 21  main.go
```

Example output on Unix. The process becomes `ls -l`; directory contents, owners, sizes, permissions, and dates depend on the working directory and operating system.

84 Signals

signal.NotifyContext cancels a context when one of the specified operating-system signals arrives. stop releases signal resources.

Example

```
package main

import (
    "context"
    "fmt"
    "os/signal"
    "syscall"
)

func main() {
    ctx, stop := signal.NotifyContext(
        context.Background(),
        syscall.SIGINT, syscall.SIGTERM,
    )
    defer stop()

    fmt.Println("waiting for signal")
    <-ctx.Done()
    fmt.Println("shutting down")
}
```

Output

```
waiting for signal
shutting down
```

Example after Ctrl+C or SIGTERM. The second line appears only after a signal arrives. How long the program waits depends on when that happens.

85 Exit

os.Exit terminates immediately with a status code and does not execute deferred calls. Returning from main is preferable for routine success.

Example

```
package main

import (
    "fmt"
    "os"
)

func main() {
    fmt.Fprintln(os.Stderr, "invalid arguments")
    os.Exit(2)
}
```

Output

```
invalid arguments
```

Example stderr output. The process exits with status code 2, which a calling shell can inspect. Deferred calls do not run when `os.Exit` executes.